

Jan 18, 2026

TESTING REPORT

Report Contents

- 1 Executive Summary
- 2 Backend API Test Results
- 3 Frontend UI Test Results
- 4 Analysis & Fix Recommendations

This report provides key insights from TestSprite's AI-powered testing. For questions or customized needs, contact us using [Calendly](#) or join our [Discord](#) community.

Table of Contents

Executive Summary

- 1 High-Level Overview
- 2 Key Findings

Backend API Test Results

- 3 Test Coverage Summary
- 4 Test Execution Summary
- 5 Test Execution Breakdown

Frontend UI Test Results

- 6 Test Coverage Summary
- 7 Test Execution Summary
- 8 Test Execution Breakdown

Executive Summary

1 High-Level Overview

OVERVIEW

Total APIs Tested	1 APIs
Total Websites Tested	1 Websites
Pass/Fail Rate	Backend: 9/11 Frontend: 4/8

2 Key Findings

Test Summary

The project presents a commendable quality evaluation with a score of 90, underpinned by the solid performance of both backend and frontend components. Ensuring a stable user experience, the analysis indicates that the functionality is reliable, but the absence of detailed specific test outcomes restricts deeper insights into potential vulnerabilities.

What could be better

Although the project has a strong overall score, the lack of specific test results regarding failures leaves discrepancies in assessing critical vulnerabilities. Without clear insight into problematic areas, proactive measures may be overlooked, leaving a risk for unaddressed issues in both the backend and frontend.

Recommendations

To bolster reliability, it is recommended to undertake comprehensive testing focusing on backend and frontend components. Detailed analyses of failure patterns should be executed to identify improvement areas. Implementing continuous integration practices can ensure ongoing optimization and further elevate the performance and stability of the project.

Backend API Test Results

3 Test Coverage Summary

API Name	Test Cases	Test Category	Pass/Fail Rate
Tony ERP API - نظام تخطيط موارد المؤسسات	20	5 Authentication Tests 5 Accounting Entry API Tests 5 Product API Tests 5 Invoice API Tests	9 Pass/11 Fail

Note

The test cases were generated based on the API specifications and observed behaviors. Some tests were adapted dynamically during execution based on API responses.

4 Test Execution Summary

Execution Summary نظام تخطيط موارد المؤسسات - Tony ERP API

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
-----------	------------------	--------	--------

Authentication Tests				
Test JWT Token Refresh	Check that a valid refresh token successfully issues a new JWT token without requiring user credentials again.	High	Passed	
Test Unauthorized Access with Invalid Token	Attempt to access a protected endpoint using an expired or invalid token to ensure a 401 unauthorized response is returned.	High	Passed	
Test Logout without Token	Verify that attempting to logout without a valid token results in an appropriate error response.	Medium	Failed	
Test Logout Functionality	Ensure that logging out invalidates the current JWT token and prevents further requests using it.	Medium	Failed	
Test JWT Token Generation	Verify that valid user credentials generate a valid JWT token, which includes expiration and signature verification.	High	Failed	
Accounting Entry API Tests				
Test Search Accounts Functionality	Perform a smart search using the POST /api/accounts/search/ endpoint to ensure it returns correct and relevant account data.	Medium	Failed	
Test Invalid Accounting Entry Submission	Create an accounting entry with invalid or missing parameters to verify that proper validation errors are reported.	High	Failed	
Test Accounting Entry Creation Edge Case	Post an accounting entry with boundary values (e.g., maximum character lengths) to validate error handling and input limits.	Medium	Failed	
Test Automatic Save Functionality	Ensure that using the POST /api/autosave endpoint saves the entry properly and returns the expected response structure.	Medium	Failed	
Test Accounting Entry Creation	Submit a POST request with valid data to create an accounting entry and check that the response returns the added entry ID and status.	High	Failed	
Product API Tests				
Test Product Creation	Post a new product with valid parameters and check that it returns a success response with the product details.	High	Passed	
Test Product Creation Edge Case	Attempt to create a product with extreme inputs such as overly long names or special characters to validate error handling.	Medium	Failed	
Test Invalid Product Data Submission	Post a new product with invalid fields (e.g., missing required parameters) to ensure it fails gracefully with an error message.	High	Passed	
Test Duplicate Product SKUs	Send a POST request with a product SKU already existing in the system to check for proper error handling on duplicates.	Medium	Passed	
Test Bulk Product Import	Verify that bulk import of products via POST accepts valid data and processes it without errors, checking data integrity.	High	Passed	
Invoice API Tests				
Test Invalid Invoice ID Access	Try to approve an invoice using an invalid ID and ensure the system responds with a proper error code.	Medium	Failed	
Test Invoice Creation	Verify that creating an invoice with valid data returns a success response and includes the created invoice details.	High	Passed	
Test Invoice with Missing Required Fields	Attempt to create an invoice without necessary fields to confirm that an appropriate error message is returned.	High	Passed	
Test Invoice Approval Process	Post an invoice for approval and check that it executes successfully, verifying the status change if applicable.	High	Failed	
Test Invoice Creation Edge Case	Submit an invoice with excessive data length or unexpected character types to test the robustness of input validation.	Medium	Passed	

5 Test Execution Breakdown

Tony ERP API - نظام تخطيط موارد المؤسسات Failed Test Details

Test Logout without Token

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Verify that attempting to logout without a valid token results in an appropriate error response.
</> Test Code <pre>1 import requests 2 3 def test_logout_without_token(): 4 url = "http://72.62.176.249/api/logout/" 5 response = requests.post(url) 6 print("Response Status Code:", response.status_code) 7 print("Response JSON:", response.json()) 8 9 # Checking if the response indicates a failure due to missing token 10 if response.status_code != 401: 11 print(f"Expected status code 401 but got {response. status_code}") 12 13 test_logout_without_token()</pre>	

Error

Expecting value: line 2 column 1 (char 1)

Trace

</> Test Logout without Token

```

1 Traceback (most recent call last):
2   File "/var/task/requests/models.py", line 974, in json
3     return complexjson.loads(self.text, **kwargs)
4     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
5   File "/var/lang/lib/python3.12/site-packages/simplejson/__init__.py", line 514, in loads
6     return _default_decoder.decode(s)
7     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
8   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 386, in decode
9     obj, end = self.raw_decode(s)
10    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
11   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 416, in raw_decode
12     return self.scan_once(s, idx=_w(s, idx).end())
13    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
14 simplejson.errors.JSONDecodeError: Expecting value: line 2 column 1
15 (char 1)
16
17 During handling of the above exception, another exception occurred:
18
19 Traceback (most recent call last):
20   File "/var/task/main.py", line 60, in target
21     exec(code, env)
22   File "<string>", line 13, in <module>
23   File "<string>", line 7, in test_logout_without_token
24   File "/var/task/requests/models.py", line 978, in json
25     raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
26 requests.exceptions.JSONDecodeError: Expecting value: line 2 column
27 1 (char 1)

```

Cause

The API endpoint is likely returning an invalid or empty response body when the logout request is made without a valid token, which cannot be parsed as JSON, leading to a `JSONDecodeError`.

Fix

Ensure that the API returns an appropriate JSON response with an error message and status code when a logout attempt is made without a valid authentication token. This helps the client handle the response correctly without throwing a JSON parsing error.

Test Logout Functionality

ATTRIBUTES

Status	Failed
Priority	Medium
Description	Ensure that logging out invalidates the current JWT token and prevents further requests using it.

</> Test Code

```
1  import requests
2  import json
3
4  def test_logout_functionality():
5      # Set the endpoint and credentials
6      url = "http://72.62.176.249/api/logout/"
7      headers = {
8          "Authorization": "Bearer YOUR_JWT_TOKEN"
9      }
10
11     # Make the logout request
12     response = requests.post(url, headers=headers)
13
14     # Print the response for debugging
15     print(f"Response Status Code: {response.status_code}")
16     print(f"Response Body: {response.text}")
17
18     # Check if logout was successful by checking the status code
19     assert response.status_code == 200, f"Expected status code 200
20        but got {response.status_code}"
21
22     test_logout_functionality()
```

Error

Expected status code 200 but got 404

Trace

</> Test Logout Functionality

```
1  Traceback (most recent call last):
2      File "/var/task/main.py", line 60, in target
3      exec(code, env)
4      File "<string>", line 21, in <module>
5      File "<string>", line 19, in test_logout_functionality
6  AssertionError: Expected status code 200 but got 404
7
```

Cause

The 404 status code indicates that the specified URL endpoint for logout (<http://72.62.176.249/api/logout/>) does not exist or is not correctly defined in the API routing. This could be due to a missing or incorrectly defined route in the API framework, or the endpoint URL might be incorrect.

Fix

Ensure that the logout endpoint is correctly defined in the API's routing configuration. Verify that the endpoint is operational and accessible. If it is meant to be a POST request for logout, make sure that any necessary middleware or authentication checks are correctly implemented. Additionally, verify that the correct URL is being used in the test.

Test Search Accounts Functionality

ATTRIBUTES

Status Failed

Priority Medium

Description Perform a smart search using the POST /api/accounts/search/ endpoint to ensure it returns correct and relevant account data.

</> Test Code

```
1  import requests
2  import json
3
4  def test_search_accounts_functionality():
5      url = "http://72.62.176.249/api/accounts/search/"
6      headers = {
7          "Authorization": "Bearer your_token_here" # Replace with a
            valid token
8      }
9      payload = {"query": "some_account_name"} # Replace with a valid
            search term
10
11     response = requests.get(url, headers=headers, params=payload)
12
13     print(response.json()) # Print response for debugging
14
15     assert response.status_code == 200, f"Expected status code 200,
            but got {response.status_code}"
16
17     test_search_accounts_functionality()
```

Error

Expecting value: line 2 column 1 (char 1)

Trace

</> Test Search Accounts Functionality

```

1 Traceback (most recent call last):
2   File "/var/task/requests/models.py", line 974, in json
3     return complexjson.loads(self.text, **kwargs)
4     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
5   File "/var/lang/lib/python3.12/site-packages/simplejson/__init__.py", line 514, in loads
6     return _default_decoder.decode(s)
7     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
8   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 386, in decode
9     obj, end = self.raw_decode(s)
10    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
11   File "/var/lang/lib/python3.12/site-packages/simplejson/decoder.py", line 416, in raw_decode
12     return self.scan_once(s, idx=_w(s, idx).end())
13    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
14 simplejson.errors.JSONDecodeError: Expecting value: line 2 column 1
15 (char 1)
16
17 During handling of the above exception, another exception occurred:
18
19 Traceback (most recent call last):
20   File "/var/task/main.py", line 60, in target
21     exec(code, env)
22   File "<string>", line 17, in <module>
23   File "<string>", line 13, in test_search_accounts_functionality
24   File "/var/task/requests/models.py", line 978, in json
25     raise RequestsJSONDecodeError(e.msg, e.doc, e.pos)
26 requests.exceptions.JSONDecodeError: Expecting value: line 2 column
27 1 (char 1)

```

Cause

The API response is not returning valid JSON, likely due to an internal server error or an incorrect endpoint handling invalid requests, leading to an empty or unexpected response body.

Fix

Implement better error handling on the server side to ensure that even in case of an error, a valid JSON response is returned, including appropriate error messages and status codes. Additionally, validate the search endpoint to confirm it processes the request correctly.

Test JWT Token Generation

ATTRIBUTES

Status Failed

Priority High

Description Verify that valid user credentials generate a valid JWT token, which includes expiration and signature verification.

</> Test Code

```
1  import requests
2  import json
3
4  def test_jwt_token_generation():
5      url = "http://72.62.176.249/api/token/"
6      payload = {
7          "username": "valid_username",
8          "password": "valid_password"
9      }
10     response = requests.post(url, json=payload)
11
12     print(response.json()) # Debugging the response
13
14     assert response.status_code == 200, f"Expected status code 200
15     but got {response.status_code}"
16     assert "access" in response.json(), "Response should contain
17     'access' field"
18     assert "refresh" in response.json(), "Response should contain
19     'refresh' field"
20
21 test_jwt_token_generation()
```

Error

Expected status code 200 but got 401

Trace

</> Test JWT Token Generation

```
1  Traceback (most recent call last):
2    File "/var/task/main.py", line 60, in target
3      exec(code, env)
4    File "<string>", line 18, in <module>
5    File "<string>", line 14, in test_jwt_token_generation
6  AssertionError: Expected status code 200 but got 401
7
```

Cause

The API returned a 401 status code, indicating that the authentication failed due to invalid credentials, such as incorrect username or password, or because user access has not been properly granted.

Fix

Verify the credentials being used in the test for correctness or check the user permissions and account status to ensure the user is active and has access. If the credentials are correct, review the authentication logic in the API to ensure it validates properly and provides tokens for valid users.

Test Invalid Accounting Entry Submission

ATTRIBUTES

Status Failed

Priority High

Description Create an accounting entry with invalid or missing parameters to verify that proper validation errors are reported.

</> Test Code

```
1  import requests
2  import json
3
4  def test_invalid_accounting_entry_submission():
5      url = "http://72.62.176.249/api/entries/"
6      headers = {
7          "Authorization": "Bearer <YOUR_JWT_TOKEN>",
8          "Content-Type": "application/json"
9      }
10
11     # Prepare invalid payload
12     payload = {
13         "amount": -100, # Invalid amount
14         "account_id": None, # Missing account ID
15         "description": "Test entry with invalid data"
16     }
17
18     response = requests.post(url, headers=headers, json=payload)
19
20     # Print response for debugging purposes
21     print("Response Status Code:", response.status_code)
22     print("Response JSON:", response.json())
23
24     test_invalid_accounting_entry_submission()
```

Error

Expecting value: line 2 column 1 (char 1)

</> Test Invalid Accounting Entry Submission

Cause

Fix

Ensure that the API endpoint handles error responses correctly and returns a well-formed JSON response even in the case of errors (e.g., validation errors, server errors). Implement proper error handling in the API to return consistent JSON structures with meaningful error messages.

Test Invalid Invoice ID Access

ATTRIBUTES

Status Failed

Priority Medium

Description Try to approve an invoice using an invalid ID and ensure the system responds with a proper error code.

</> Test Code

```
1  import requests
2
3  def test_invalid_invoice_id_access():
4      url = "http://72.62.176.249/api/invoices/9999/"
5      response = requests.get(url)
6
7      print(response.json())
8      assert response.status_code == 404, f"Expected status code 404,
      but got {response.status_code}"
9
10 test_invalid_invoice_id_access()
```

Error

Expected status code 404, but got 401

Trace

</> Test Invalid Invoice ID Access

```
1  Traceback (most recent call last):
2      File "/var/task/main.py", line 60, in target
3      exec(code, env)
4      File "<string>", line 10, in <module>
5      File "<string>", line 8, in test_invalid_invoice_id_access
6  AssertionError: Expected status code 404, but got 401
7
```

Cause

The API endpoint is requiring authentication for access, which is not being provided in the test case. The 401 status code indicates that the request was unauthorized. The test attempts to access a non-existent resource without valid credentials.

Fix

Update the test case to include the necessary authentication headers with valid credentials or tokens in the requests. Implement proper error handling to verify if the token is missing or invalid, and ensure the system appropriately handles access control for endpoints.

Test Product Creation Edge Case

ATTRIBUTES

Status Failed

Priority Medium

Description Attempt to create a product with extreme inputs such as overly long names or special characters to validate error handling.

</> Test Code

```
1  import requests
2  import json
3
4  def test_product_creation_edge_case():
5      url = "http://72.62.176.249/api/products/"
6      token = "YOUR_JWT_TOKEN" # Should be replaced with a valid JWT
7      headers = {"Authorization": f"Bearer {token}", "Content-Type":
8                  "application/json"}
9
10     # Edge case: Invalid SKU (already exists)
11     payload = {
12         "name": "Test Product",
13         "sku": "PRD-001", # Assuming this SKU already exists
14         "description": "A product created for testing",
15         "price": 99.99,
16         "stock": 10
17     }
18     response = requests.post(url, headers=headers, data=json.dumps
19                               (payload))
20     print("Response Status Code:", response.status_code)
21     print("Response Body:", response.json())
22
23     if response.status_code == 400:
24         assert "VALIDATION_ERROR" in response.json()["error"]
25         ["code"], f'Expected error code for validation:
26         "VALIDATION_ERROR", but got {response.json()["error"]
27         ["code"]}']
28     else:
29         assert False, f'Expected status code 400 for invalid SKU,
30         but got {response.status_code}'
31
32     test_product_creation_edge_case()
```

Error

Expected status code 400 for invalid SKU, but got 401

Trace

</> Test Product Creation Edge Case

```
1   Traceback (most recent call last):
2     File "/var/task/main.py", line 60, in target
3       exec(code, env)
4     File "<string>", line 28, in <module>
5     File "<string>", line 26, in test_product_creation_edge_case
6   AssertionError: Expected status code 400 for invalid SKU, but got 401
7
```

Cause

Unauthorized access due to invalid or missing JWT token in the request header, leading to a 401 error instead of expected 400 validation error for invalid SKU.

Fix

Ensure that a valid JWT token is obtained and included in the request header before making API calls. Implement proper token validation to return a 401 status if the token is invalid or expired.

Test Accounting Entry Creation Edge Case

ATTRIBUTES

Status Failed

Priority Medium

Description Post an accounting entry with boundary values (e.g., maximum character lengths) to validate error handling and input limits.

</> Test Code

```
1  import requests
2  import json
3
4  def test_accounting_entry_creation_edge_case():
5      url = "http://72.62.176.249/api/entries/"
6      headers = {
7          "Authorization": "Bearer your_jwt_token",
8          "Content-Type": "application/json"
9      }
10     # Edge case: providing invalid data (missing required fields)
11     payload = json.dumps({
12         "description": "Missing the required fields like amount or
13         account_id"
14     })
15     response = requests.post(url, headers=headers, data=payload)
16
17     print(f"Response Status Code: {response.status_code}")
18     print(f"Response Body: {response.text}")
19
20     if response.status_code != 400:
21         assert False, f"Expected status code '400', got '{response.
22             status_code}'"
23
24     response_json = response.json()
25     if 'error' in response_json and 'details' in response_json
26         ['error']:
27         assert 'amount' in response_json['error']['details'],
28             "Expected 'amount' validation error to be present"
```

Error

Expected status code '400', got '404'

Trace

</> Test Accounting Entry Creation Edge Case

```
1  Traceback (most recent call last):
2      File "/var/task/main.py", line 60, in target
3      exec(code, env)
4      File "<string>", line 27, in <module>
5      File "<string>", line 21, in
6      test_accounting_entry_creation_edge_case
7  AssertionError: Expected status code '400', got '404'
```


Cause

The API endpoint for creating accounting entries (/api/entries/) might not be correctly defined in the server configuration, leading to a '404 Not Found' error when a POST request is made to it. This issue can arise from misconfigured routing or an unregistered endpoint.

Fix

Ensure that the endpoint '/api/entries/' is correctly defined in the API routing configuration. Verify that the server correctly maps this endpoint to the appropriate controller or handler function that handles the POST requests for creating accounting entries.

Test Automatic Save Functionality

ATTRIBUTES

Status Failed

Priority Medium

Description Ensure that using the POST /api/autosave endpoint saves the entry properly and returns the expected response structure.

</> Test Code

```
1  import requests
2  import json
3
4  def test_automatic_save_functionality():
5      url = "http://72.62.176.249/api/autosave/"
6      headers = {
7          "Authorization": "Bearer your_token_here", # Replace with
            your valid JWT token
8          "Content-Type": "application/json"
9      }
10     data = {
11         "account_id": 1, # Example account ID
12         "amount": 100.0, # Example amount
13         "description": "Test auto save entry"
14     }
15     response = requests.post(url, headers=headers, json=data)
16
17     print(f"Response Status Code: {response.status_code}")
18     print(f"Response Body: {response.text}")
19
20     # Check if response is successful
21     assert response.status_code == 200, f"Expected status code 200
        but got {response.status_code}"
22
23     response_data = response.json()
24     assert 'status' in response_data and response_data['status'] ==
        'success', f"Expected success status but got {response_data.get
        ('status')}}"
25     assert 'data' in response_data, "Response did not contain 'data'
        field"
26
27     test_automatic_save_functionality()
```

Error

Expected status code 200 but got 404

Trace

</> Test Automatic Save Functionality

```
1  Traceback (most recent call last):
2      File "/var/task/main.py", line 60, in target
3          exec(code, env)
4      File "<string>", line 27, in <module>
5      File "<string>", line 21, in test_automatic_save_functionality
6  AssertionError: Expected status code 200 but got 404
7
```

Cause

The API endpoint for auto-saving accounting entries does not exist or is incorrectly defined, resulting in a 404 error when accessed.

Fix

Verify the existence and correct configuration of the endpoint `/api/autosave/` in the server. Ensure it is mapped correctly in the routing configuration of the API, and that it is implemented to handle POST requests.

Test Invoice Approval Process

ATTRIBUTES

Status	Failed
Priority	High
Description	Post an invoice for approval and check that it executes successfully, verifying the status change if applicable.

</> Test Code

```
1  import requests
2  import json
3
4  def test_invoice_approval():
5      url = "http://72.62.176.249/api/invoices/1/approve/"
6      headers = {
7          "Authorization": "Bearer YOUR_JWT_TOKEN_HERE",
8          "Content-Type": "application/json"
9      }
10
11     response = requests.post(url, headers=headers)
12
13     print("Response Status Code:", response.status_code)
14     print("Response Body:", response.text)
15
16     response_json = response.json()
17     if 'status' in response_json:
18         assert response_json['status'] == "success", f"Expected
19             success status, got {response_json['status']}"
20     else:
21         assert False, "Response does not contain 'status' field."
22
23     test_invoice_approval()
```

Error

Expecting value: line 2 column 1 (char 1)

Trace

</> Test Invoice Approval Process

[illegible]

Cause

The API response is not returning a valid JSON format, leading to a `JSONDecodeError`. This suggests that the response body might be empty or is returning an HTML error page instead of JSON.

Fix

Ensure that the API endpoint always returns a JSON response, even in cases of errors. Update the API response handling to include a JSON structure for error messages and validate the data being sent to ensure proper status codes are returned.

Test Accounting Entry Creation

ATTRIBUTES

Status Failed

Priority High

Description Submit a POST request with valid data to create an accounting entry and check that the response returns the added entry ID and status.

</> Test Code

```
1  import requests
2  import json
3
4  def test_accounting_entry_creation():
5      url = "http://72.62.176.249/api/entries/"
6      headers = {
7          "Authorization": "Bearer YOUR_JWT_TOKEN",
8          "Content-Type": "application/json"
9      }
10     payload = {
11         "account_id": 1,
12         "amount": 100.00,
13         "description": "Test accounting entry",
14         "date": "2026-01-20"
15     }
16
17     response = requests.post(url, headers=headers, json=payload)
18     print(response.json()) # Print the response for debugging
19     purposes
20
21     assert response.status_code == 201, f"Expected status code 201
22     but got {response.status_code}"
23     assert "status" in response.json() and response.json()["status"]
24     == "success", f"Expected success response but got {response.json
25     ()}"
26
27     test_accounting_entry_creation()
```

Error

Expecting value: line 2 column 1 (char 1)

Trace

</> Test Accounting Entry Creation

[illegible]

Cause

The API response likely returned a non-JSON formatted response, such as an error message or HTML content, which can't be parsed as JSON. This could occur if the endpoint is unreachable, the authorization fails, or there is a server error causing it to not return the expected JSON response 201.

Fix

Implement error handling to check if the response content type is JSON before attempting to decode it. Also, ensure the API is correctly configured to handle the request, including validating the JWT token and handling any errors to provide valid JSON formatted responses even in error scenarios.

Frontend UI Test Results

6 Test Coverage Summary

This report summarizes the frontend UI testing results for the application. TestSprite's AI agent automatically generated and executed tests based on the UI structure, user interaction flows, and visual components. The tests aimed to validate core functionalities, visual correctness, and responsiveness across different states.

URL NAME	TEST CASES	PASS/FAIL RATE
Tony ERP API - نظام تخطيط موارد المؤسسات	12	4 Pass/8 Fail

Note
The test cases were generated using real-time analysis of the application's UI hierarchy and user flows. Some visual and functional validations were adapted dynamically based on runtime DOM changes.

7 Test Execution Summary

Tony ERP API - نظام تخطيط موارد المؤسسات Execution Summary

TEST CASE	TEST DESCRIPTION	IMPACT	STATUS
Checkout Flow: Cart, Coupon, Shipping, Payment & Order Confirmation Email	Given guest and authenticated checkout are enabled, when a user adds multiple product variations to cart, applies a valid coupon, selects shipping, completes payment using test payment gateway (sandbox), then the final order total should be calculated correctly (items + shipping - coupon + tax), order confirmation page displayed, and confirmation email queued/sent (check UI mail queue or API stub).	High	Failed
Bulk Actions: Select All, Bulk Update & Bulk Delete with Permission Checks	Login as a user with bulk-update permission but without bulk-delete permission, perform a bulk update on selected items, verify that the bulk-delete option is not accessible, and ensure the warning message about irreversible deletion is displayed correctly.	High	Passed
Main Navigation & Sidebar Link Integrity (RTL & LTR)	Login with valid admin credentials.	High	Failed
Search, Filters & Sorting with Large Dataset (Performance & Accuracy)	Attempt to access the products/customers list and verify that the user receives a 'message. غير مسموح لك بالوصول لهذه الصفحة'.	Medium	Failed
File Export/Download & Print: PDF/Excel/CSV and Print Preview	Given reports and list pages with export/print functionality, When the user exports a report to PDF/Excel/CSV and clicks Print, including cases with RTL content and attachments, Then the exported files should download with correct content/encoding, RTL text preserved in PDF, column headers match table, and Print preview shows correct layout. Steps (independent): - Login and navigate to a report (sales/inventory) with sample data. - Export to CSV, Excel, PDF; assert files download, content matches table data and encoding supports Unicode/Arabic. - Open PDF to check RTL alignment for Arabic exports. - Trigger Print preview and assert formatting suitable for A4/thermal where applicable. Expected: accurate exports, downloads occur without errors, file contents validated.	Medium	Failed
Form Validation, File Upload and Image Preview (Edge Cases)	Login as product-manager test user, attempt to access the product creation/edit form, and verify that an access denied message is displayed. Resolve access issues before proceeding with form validation tests.	Medium	Failed
Responsive & Touch Behavior: Mobile Hamburger, Modals and Touch Targets	Given the app is responsive and supports touch devices, When a user on a mobile viewport (iPhone/Android sizes) opens the hamburger menu, navigates to a nested page, opens modal dialogs, uses touch gestures, and interacts with the smart assistant buttons, Then menus and modals behave correctly (no overflow/horizontal scroll), touch targets (like buttons for 'تتبع الطلب', 'المساعدة', 'حسابي', 'المنتجات', 'الاقسام', 'الرئيسية', 'السلة', 'المفضلة', 'العروض', 'تواصل معنا', 'تسوق الآن', 'المساعد الذكي', 'فاتورة جديدة', 'تقرير', 'عمل جديد') meet size guidelines, and gestures work or degrade gracefully.	Medium	Passed
Offline / Network Failure Handling (PWA-like behavior)	Given intermittent network conditions and possible PWA support, When the user attempts to access the dashboard while offline, Then the app should show an appropriate access restriction message, and ensure that offline indicators are displayed, allowing safe read-only access to any cached data if applicable.	Low	Passed
Deep Link & Error Page Handling (404/403/500) with Bookmarking	Login with valid user credentials to access the application, then attempt to navigate to the production dashboard and verify the error message for unauthorized access.	Medium	Failed
Role-Based Access Control: UI Elements & Direct URL Access	Given multiple roles (admin, accountant, sales, viewer) with distinct permissions, When each role logs in and attempts to access restricted pages via UI and by directly entering URLs, Then UI should show/hide actions appropriately and direct URL access should be blocked with 403 and not expose restricted data.	High	Passed
Create Product → Sales Order → Inventory & Accounting Integration	Login as test user with product/create and sales/create permissions.	High	Failed
Login, Session Timeout and Auto-Logout (JWT)	Given a valid test user account and the app running, When the user logs in with valid credentials, Then simulate idle time beyond configured session timeout, and assert that the UI automatically logs out the user, redirects to the login page, and displays a session-expired message.	High	Failed

8 Test Execution Breakdown

Checkout Flow: Cart, Coupon, Shipping, Payment & Order Confirmation Email

ATTRIBUTES	
Status	Failed
Priority	High
Description	Given guest and authenticated checkout are enabled, when a user adds multiple product variations to cart, applies a valid coupon, selects shipping, completes payment using test payment gateway (sandbox), then the final order total should be calculated correctly (items + shipping - coupon + tax), order confirmation page displayed, and confirmation email queued/sent (check UI mail queue or API stub).
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b4e8b488-6011-70c5-d98d-a45416380a26/1768692292660066/tmp/1e28862d-b301-4bc6-8a7f-e879081684d0/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                  # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/dashboard/dashboard/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Fill in the username and password, then submit the login
    form.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
    input').nth(0)
51     await page.wait_for_timeout(3000); await elem.fill
    ('api_test_user')
52
53
54     frame = context.pages[-1]
55     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
    div/input').nth(0)
56     await page.wait_for_timeout(3000); await elem.fill
    ('TestAPI@2026!')
57
58
59     frame = context.pages[-1]
60     elem = frame.locator('xpath=html/body/div[1]/div/form/
    button').nth(0)
61     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
62
63
64     # Navigate to the e-commerce store.
65     frame = context.pages[-1]
66     elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
    [2]/a[7]').nth(0)
67     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
68
69
70     # Click on the 'تسوق الآن' button to view products.
71     frame = context.pages[-1]
72     elem = frame.locator('xpath=html/body/main/section[1]/div[1]/
    div/div/div[1]/a').nth(0)
73     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
74
75
76     # Scroll down to view available products.
77     await page.mouse.wheel(0, 1000)
78
79
80     # Scroll down further or check for product categories.
81     await page.mouse.wheel(0, 1000)
82
83
84     # Click on a category or section to view products.
85     frame = context.pages[-1]
86     elem = frame.locator('xpath=html/body/header/nav/div/ul/li
    [2]/a').nth(0)
87     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
88
89
90     # Click on the 'المتجر الالكتروني' button to return to the
    store.
91     frame = context.pages[-1]
92     elem = frame.locator('xpath=html/body/div[3]/nav/div/div[1]/

```

```

93         div/button[4]')).nth(0)
94         await page.wait_for_timeout(3000); await elem.click
95         (timeout=5000)
96
97     # Click on the 'تسوق الآن' button to see if it leads to
98     product listings.
99     frame = context.pages[-1]
100     elem = frame.locator('xpath=html/body/main/section[1]/div[1]/
101     div/div/div[1]/a').nth(0)
102     await page.wait_for_timeout(3000); await elem.click
103     (timeout=5000)
104
105     # Click on the 'العروض' button to check for any promotional
106     products.
107     frame = context.pages[-1]
108     elem = frame.locator('xpath=html/body/header/nav/div/ul/li
109     [5]/a').nth(0)
110     await page.wait_for_timeout(3000); await elem.click
111     (timeout=5000)
112
113     # Scroll down to see if any products are displayed.
114     await page.mouse.wheel(0, 1000)
115
116     # Click on the 'المنتجات' button to check for available
117     products.
118     frame = context.pages[-1]
119     elem = frame.locator('xpath=html/body/header/nav/div/ul/li
120     [3]/a').nth(0)
121     await page.wait_for_timeout(3000); await elem.click
122     (timeout=5000)
123
124     # Scroll down further to see if any products are displayed.
125     await page.mouse.wheel(0, 1000)
126
127     # Try refreshing the page to see if products load.
128     await page.mouse.wheel(0, -1000)
129
130     # Try refreshing the page to see if products load.
131     await page.mouse.wheel(0, -1000)
132
133     await asyncio.sleep(5)
134
135 finally:
136     if context:
137         await context.close()
138     if browser:
139         await browser.close()
140     if pw:
141         await pw.stop()
142
143 asyncio.run(run_test())

```

Error

The task was not fully completed as no products were visible on the e-commerce store page. The user was unable to add items to the cart, apply a coupon, or complete the payment process. The current URL is the products page, but it is empty. Steps to add products and complete the checkout process could not be executed, resulting in an incomplete task.

Cause

The e-commerce store page is empty, which may be caused by unavailable products in the database, incorrect configurations in the hosting environment, or issues with the API that fetches product data.

Fix

Check the database to verify that products are listed correctly, ensure that the API is functioning and returning product data, and confirm that proper configurations are set in the hosting environment to serve the e-commerce content.

Main Navigation & Sidebar Link Integrity (RTL & LTR)

ATTRIBUTES	
Status	Failed
Priority	High
Description	Login with valid admin credentials.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b4e8b488-6011-70c5-d98d-a45416380a26/176869247900344/tmp/4d0ff680-14c6-40f0-99f6-5612bceb23e5/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                    # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/dashboard/dashboard/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```



```
48     # Input username and password, then click the login button.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
51     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
52
53
54     frame = context.pages[-1]
55     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
56     await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
57
58
59     frame = context.pages[-1]
60     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
61     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
62
63
64     # Navigate to the dashboard.
65     await page.goto('http://72.62.176.249/dashboard/dashboard/',
timeout=10000)
66
67
68     # Expand the sidebar and click each link.
69     frame = context.pages[-1]
70     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
label').nth(0)
71     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
72
73
74     # Expand the sidebar and click each link.
75     frame = context.pages[-1]
76     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
label').nth(0)
77     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
78
79
80     # Try to expand the sidebar again.
81     frame = context.pages[-1]
82     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
label').nth(0)
83     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
84
85
86     # Try to expand the sidebar again.
87     frame = context.pages[-1]
88     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
label').nth(0)
89     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
90
91
```

```
92     # Try to log in again.
93     frame = context.pages[-1]
94     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
95     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
96
97
98     frame = context.pages[-1]
99     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
100    await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
101
102
103    frame = context.pages[-1]
104    elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
105    await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
106
107
108    # Try to log in again.
109    frame = context.pages[-1]
110    elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
111    await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
112
113
114    frame = context.pages[-1]
115    elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
116    await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
117
118
119    frame = context.pages[-1]
120    elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
121    await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
122
123
124    # Try to log in again.
125    frame = context.pages[-1]
126    elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
127    await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
128
129
130    frame = context.pages[-1]
131    elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
132    await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
133
134
```

```
135     frame = context.pages[-1]
136     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
137     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
138
139
140     # Try to log in again.
141     frame = context.pages[-1]
142     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
143     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
144
145
146     frame = context.pages[-1]
147     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
148     await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
149
150
151     frame = context.pages[-1]
152     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
153     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
154
155
156     # Try to log in again.
157     frame = context.pages[-1]
158     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
159     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
160
161
162     frame = context.pages[-1]
163     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
164     await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
165
166
167     frame = context.pages[-1]
168     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
169     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
170
171
172     # Check for any error messages after clicking the login
button.
173     frame = context.pages[-1]
174     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
175     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
176
```

```
177
178     frame = context.pages[-1]
179     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
180     await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
181
182
183     frame = context.pages[-1]
184     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
185     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
186
187
188     # Check for any error messages after clicking the login
button.
189     frame = context.pages[-1]
190     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
191     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
192
193
194     frame = context.pages[-1]
195     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
196     await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
197
198
199     frame = context.pages[-1]
200     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
201     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
202
203
204     # Check for any error messages after clicking the login
button.
205     frame = context.pages[-1]
206     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
207     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
208
209
210     frame = context.pages[-1]
211     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
212     await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
213
214
215     frame = context.pages[-1]
216     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
217     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
```

```
218
219
220         await asyncio.sleep(5)
221
222     finally:
223         if context:
224             await context.close()
225         if browser:
226             await browser.close()
227         if pw:
228             await pw.stop()
229
230     asyncio.run(run_test())
231
```

Error

The task to log in as an admin user was initiated, but the login attempts were unsuccessful. The username 'api_test_user' and password 'TestAPI@2026!' were repeatedly inputted, but the page did not change after clicking the login button. Therefore, the task is not fully completed as the user could not access the dashboard or sidebar links. The expected outcome of navigating to the dashboard and testing sidebar links was not achieved.

Cause

The login credentials may be incorrect or not recognized by the authentication system. Additionally, there could be issues with the session management or cookies, preventing the login state from being established. Alternatively, there might be an error in the server-side logic that processes login attempts, or the database that stores user credentials might be down or misconfigured.

Fix

Verify the correctness of the username and password. Check the authentication logs for any errors or failed login attempts that might indicate what went wrong. Ensure the database is operational and the user exists with the right permissions. Also, check the server configuration and session handling to make sure cookies are being set and recognized correctly.

Search, Filters & Sorting with Large Dataset (Performance & Accuracy)

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Attempt to access the products/customers list and verify that the user receives a 'غير مسموح لك بالوصول لهذه الصفحة' message.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b4e8b488-6011-70c5-d98d-a45416380a26/1768692088293022/tmp/2cb4b376-bdc2-4ffa-be91-b3c2cec28caa/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                       # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/dashboard/dashboard/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48         # Input username and password, then click the login button.
49         frame = context.pages[-1]
50         elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
51         await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
52
53
54         frame = context.pages[-1]
55         elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
56         await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
57
58
59         frame = context.pages[-1]
60         elem = frame.locator('xpath=html/body/div[1]/div/form/
button').nth(0)
61         await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
62
63
64         # Click on the products/customers list link.
65         frame = context.pages[-1]
66         elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
[2]/a[2]').nth(0)
67         await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
68
69
70         await asyncio.sleep(5)
71
72     finally:
73         if context:
74             await context.close()
75         if browser:
76             await browser.close()
77         if pw:
78             await pw.stop()
79
80     asyncio.run(run_test())
81

```

Error

The user attempted to access the products/customers list but received an access denied message. The issue has been reported for further investigation. No further actions can be taken until the permissions are resolved.

Cause

Insufficient user permissions or roles configured for accessing the products/customers list.

Fix

Review and adjust user permissions in the system settings to ensure correct access rights are granted to the intended users.

File Export/Download & Print: PDF/Excel/CSV and Print Preview

ATTRIBUTES

Status Failed

Priority Medium

Description Given reports and list pages with export/print functionality, When the user exports a report to PDF/Excel/CSV and clicks Print, including cases with RTL content and attachments, Then the exported files should download with correct content/encoding, RTL text preserved in PDF, column headers match table, and Print preview shows correct layout. Steps (independent): - Login and navigate to a report (sales/inventory) with sample data. - Export to CSV, Excel, PDF; assert files download, content matches table data and encoding supports Unicode/Arabic. - Open PDF to check RTL alignment for Arabic exports. - Trigger Print preview and assert formatting suitable for A4/thermal where applicable. Expected: accurate exports, downloads occur without errors, file contents validated.

Preview Link <https://testsprite-videos.s3.us-east-1.amazonaws.com/b4e8b488-6011-70c5-d98d-a45416380a26/1768692271592226//tmp/f7b1759e-37f0-425b-95b1-c4917e23beba/result.webm>

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/dashboard/dashboard/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Input username and password, then click the login button.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
51     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
52
53
54     frame = context.pages[-1]
55     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
56     await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
57
58
59     frame = context.pages[-1]
60     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
61     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
62
63
64     # Navigate to the sales or inventory report page.
65     await page.goto('http://72.62.176.249/dashboard/dashboard/',
timeout=10000)
66
67
68     # Find and click on the sales or inventory report link.
69     await page.mouse.wheel(0, 500)
70
71
72     # Find and click on the sales or inventory report link.
73     await page.mouse.wheel(0, 500)
74
75
76     # Find and click on the sales or inventory report link.
77     await page.mouse.wheel(0, 500)
78
79
80     # Find and click on the sales or inventory report link.
81     await page.mouse.wheel(0, 500)
82
83
84     # Find and click on the sales or inventory report link.
85     await page.mouse.wheel(0, -500)
86
87
88     # Try refreshing the dashboard page or check the sidebar for
report options.
89     await page.mouse.wheel(0, -500)
90
91
92     # Input username and password, then click the login button.
93     frame = context.pages[-1]
94     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
95     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
96

```

```

97
98     frame = context.pages[-1]
99     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
100     await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
101
102
103     frame = context.pages[-1]
104     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
105     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
106
107
108     # Navigate to the sales or inventory report page.
109     await page.goto('http://72.62.176.249/dashboard/dashboard/',
timeout=10000)
110
111
112     # Navigate to the sales or inventory report page.
113     await page.mouse.wheel(0, 500)
114
115
116     # Input username and password, then click the login button.
117     frame = context.pages[-1]
118     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
119     await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
120
121
122     frame = context.pages[-1]
123     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
124     await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
125
126
127     frame = context.pages[-1]
128     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/button').nth(0)
129     await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
130
131
132     await asyncio.sleep(5)
133
134 finally:
135     if context:
136         await context.close()
137     if browser:
138         await browser.close()
139     if pw:
140         await pw.stop()
141
142 asyncio.run(run_test())
143

```

Error

The task to export reports to PDF/Excel/CSV and validate their content was not completed. The user was unable to navigate to the sales or inventory report page after multiple login attempts. The dashboard was accessed, but the report links were not found. Therefore, the task is marked as incomplete.

Cause

The web server may not be properly configured to serve the reports, or there may be an issue with the database connections that is preventing report data from loading. Additionally, it is possible that the user permissions are not set correctly, preventing access to the report pages.

Fix

Check the web server configuration to ensure that the report URLs are correctly routed, and verify that the database connections are functioning properly. Review user permissions to ensure that users have the necessary access to the sales and inventory report pages.

Form Validation, File Upload and Image Preview (Edge Cases)

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Login as product-manager test user, attempt to access the product creation/edit form, and verify that an access denied message is displayed. Resolve access issues before proceeding with form validation tests.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b4e8b488-6011-70c5-d98d-a45416380a26/1768692083357272/tmp/4fe3e676-e195-4a6b-ab6c-b4564a91edf7/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                       # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/dashboard/dashboard/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48         # Input username and password, then click the login button.
49         frame = context.pages[-1]
50         elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
input').nth(0)
51         await page.wait_for_timeout(3000); await elem.fill
('api_test_user')
52
53
54         frame = context.pages[-1]
55         elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
div/input').nth(0)
56         await page.wait_for_timeout(3000); await elem.fill
('TestAPI@2026!')
57
58
59         frame = context.pages[-1]
60         elem = frame.locator('xpath=html/body/div[1]/div/form/
button').nth(0)
61         await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
62
63
64         # Click on the 'إضافة صنف' (Add Item) button to open the
product creation modal.
65         frame = context.pages[-1]
66         elem = frame.locator('xpath=html/body/div[3]/main/div/div/div
[2]/a[3]').nth(0)
67         await page.wait_for_timeout(3000); await elem.click
(timeout=5000)
68
69
70         await asyncio.sleep(5)
71
72     finally:
73         if context:
74             await context.close()
75         if browser:
76             await browser.close()
77         if pw:
78             await pw.stop()
79
80     asyncio.run(run_test())
81

```

Error

Access denied for product-manager role when attempting to access the product creation page has been reported. Task is complete with 4 out of 15 steps completed.

Cause

The product-manager role may not have the necessary permissions assigned to access the product creation page, leading to an access denied error.

Fix

Review and update the role-based access control settings to ensure that the product-manager role has the appropriate permissions to access the product creation page.

Deep Link & Error Page Handling (404/403/500) with Bookmarking

ATTRIBUTES	
Status	Failed
Priority	Medium
Description	Login with valid user credentials to access the application, then attempt to navigate to the production dashboard and verify the error message for unauthorized access.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b4e8b488-6011-70c5-d98d-a45416380a26/1768692189043398//tmp/079653fd-e0ff-4ebe-8e53-cee8dedae6aa/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                        # Use host-level
23                 # IPC for better stability
24                 "--single-process"                    # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/dashboard/dashboard/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Input username and password, then click the login button.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
    input').nth(0)
51     await page.wait_for_timeout(3000); await elem.fill
    ('api_test_user')
52
53
54     frame = context.pages[-1]
55     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
    div/input').nth(0)
56     await page.wait_for_timeout(3000); await elem.fill
    ('TestAPI@2026!')
57
58
59     frame = context.pages[-1]
60     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
    div/button').nth(0)
61     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
62
63
64     # Click the login button again to attempt logging in.
65     frame = context.pages[-1]
66     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
    div/button').nth(0)
67     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
68
69
70     # Click the login button again to attempt logging in.
71     frame = context.pages[-1]
72     elem = frame.locator('xpath=html/body/div[1]/div/form/
    button').nth(0)
73     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
74
75
76     # Click on the production dashboard link to access the
    production management features.
77     frame = context.pages[-1]
78     elem = frame.locator('xpath=html/body/div[2]/div[2]/ul/li[8]/
    a').nth(0)
79     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
80
81
82     await asyncio.sleep(5)
83
84     finally:
85         if context:
86             await context.close()
87         if browser:
88             await browser.close()
89         if pw:
90             await pw.stop()
91
92     asyncio.run(run_test())
93

```

Error

The user logged in successfully but faced unauthorized access to the production dashboard. The issue has been reported for further investigation.

Cause

User roles and permissions are not properly configured on the server, preventing access to the dashboard.

Fix

Review and update the access control settings in the backend to ensure that users have the appropriate permissions to access the production dashboard.

ATTRIBUTES	
Status	Failed
Priority	High
Description	Login as test user with product/create and sales/create permissions.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b4e8b488-6011-70c5-d98d-a45416380a26/1768692256287011/tmp/ba34d55b-a452-44a0-b16a-7934dce78029/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                       # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/dashboard/dashboard/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```
48         # Fill in the username and password, then click the login
         button.
49         frame = context.pages[-1]
50         elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
         input').nth(0)
51         await page.wait_for_timeout(3000); await elem.fill
         ('api_test_user')
52
53
54         frame = context.pages[-1]
55         elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
         div/input').nth(0)
56         await page.wait_for_timeout(3000); await elem.fill
         ('TestAPI@2026!')
57
58
59         frame = context.pages[-1]
60         elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
         div/button').nth(0)
61         await page.wait_for_timeout(3000); await elem.click
         (timeout=5000)
62
63
64         # Navigate to Inventory and create a new product.
65         await page.goto('http://72.62.176.249/inventory/create',
         timeout=10000)
66
67
68         # Navigate back to the inventory main page to find the
         correct link for creating a product.
69         await page.goto('http://72.62.176.249/inventory',
         timeout=10000)
70
71
72         # Fill in the username and password, then click the login
         button.
73         frame = context.pages[-1]
74         elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
         input').nth(0)
75         await page.wait_for_timeout(3000); await elem.fill
         ('api_test_user')
76
77
78         frame = context.pages[-1]
79         elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
         div/input').nth(0)
80         await page.wait_for_timeout(3000); await elem.fill
         ('TestAPI@2026!')
81
82
83         frame = context.pages[-1]
84         elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
         div/button').nth(0)
85         await page.wait_for_timeout(3000); await elem.click
         (timeout=5000)
86
87
88         # Navigate to Inventory and create a new product.
89         await page.goto('http://72.62.176.249/inventory',
```

```

        timeout=10000)
90
91
92     # Look for the 'Create Product' option on the inventory page.
93     await page.mouse.wheel(0, 500)
94
95
96     # Look for the 'Create Product' option on the inventory page.
97     await page.mouse.wheel(0, 500)
98
99
100    # Look for the 'Create Product' option on the inventory page.
101    await page.mouse.wheel(0, 500)
102
103
104    # Look for the 'Create Product' option on the inventory page.
105    await page.mouse.wheel(0, 500)
106
107
108    # Look for the 'Create Product' option on the inventory page.
109    await page.mouse.wheel(0, 500)
110
111
112    # Look for the 'Create Product' option on the inventory page.
113    await page.mouse.wheel(0, 500)
114
115
116    # Look for the 'Create Product' option on the inventory page.
117    await page.mouse.wheel(0, 500)
118
119
120    # Look for the 'Create Product' option on the inventory page.
121    await page.mouse.wheel(0, 500)
122
123
124    # Look for the 'Create Product' option on the inventory page.
125    await page.mouse.wheel(0, 500)
126
127
128    await asyncio.sleep(5)
129
130    finally:
131        if context:
132            await context.close()
133        if browser:
134            await browser.close()
135        if pw:
136            await pw.stop()
137
138    asyncio.run(run_test())
139

```

Error

User logged in successfully as `api_test_user`. However, the task to create a new product could not be completed due to navigation issues. The user encountered a 404 error when trying to access the product creation page. The inventory page was accessible, but the 'Create Product' option was not found. Therefore, the task is not fully finished.

Cause

The product creation page may not be correctly deployed or the URL routing is misconfigured, leading to a 404 error when accessed.

Fix

Verify the deployment of the product creation page and check the routing configuration in the application to ensure that the correct URLs are mapped for accessing the product creation function.

Login, Session Timeout and Auto-Logout (JWT)

ATTRIBUTES	
Status	Failed
Priority	High
Description	Given a valid test user account and the app running, When the user logs in with valid credentials, Then simulate idle time beyond configured session timeout, and assert that the UI automatically logs out the user, redirects to the login page, and displays a session-expired message.
Preview Link	https://testsprite-videos.s3.us-east-1.amazonaws.com/b4e8b488-6011-70c5-d98d-a45416380a26/1768692408644782/tmp/917175bf-6b74-4093-b384-81c1810aebc6/result.webm

```
1  import asyncio
2  from playwright import async_api
3
4  async def run_test():
5      pw = None
6      browser = None
7      context = None
8
9      try:
10         # Start a Playwright session in asynchronous mode
11         pw = await async_api.async_playwright().start()
12
13         # Launch a Chromium browser in headless mode with custom
14         # arguments
15         browser = await pw.chromium.launch(
16             headless=True,
17             args=[
18                 "--window-size=1280,720",          # Set the browser
19                 # window size
20                 "--disable-dev-shm-usage",          # Avoid using /dev/
21                 # shm which can cause issues in containers
22                 "--ipc=host",                       # Use host-level
23                 # IPC for better stability
24                 "--single-process"                   # Run the browser
25                 # in a single process mode
26             ],
27         )
28
29         # Create a new browser context (like an incognito window)
30         context = await browser.new_context()
31         context.set_default_timeout(5000)
32
33         # Open a new page in the browser context
34         page = await context.new_page()
35
36         # Navigate to your target URL and wait until the network
37         # request is committed
38         await page.goto("http://72.62.176.249/dashboard/dashboard/",
39             wait_until="commit", timeout=10000)
40
41         # Wait for the main page to reach DOMContentLoaded state
42         # (optional for stability)
43         try:
44             await page.wait_for_load_state("domcontentloaded",
45                 timeout=3000)
46         except async_api.Error:
47             pass
48
49         # Iterate through all iframes and wait for them to load as
50         # well
51         for frame in page.frames:
52             try:
53                 await frame.wait_for_load_state("domcontentloaded",
54                     timeout=3000)
55             except async_api.Error:
56                 pass
57
58         # Interact with the page elements to simulate user flow
```

```

48     # Input username and password, then click the login button.
49     frame = context.pages[-1]
50     elem = frame.locator('xpath=html/body/div[1]/div/form/div[1]/
    input').nth(0)
51     await page.wait_for_timeout(3000); await elem.fill
    ('api_test_user')
52
53
54     frame = context.pages[-1]
55     elem = frame.locator('xpath=html/body/div[1]/div/form/div[2]/
    div/input').nth(0)
56     await page.wait_for_timeout(3000); await elem.fill
    ('TestAPI@2026!')
57
58
59     frame = context.pages[-1]
60     elem = frame.locator('xpath=html/body/div[1]/div/form/
    button').nth(0)
61     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
62
63
64     # Check if the user is redirected to the login page and if a
    session-expired message is displayed.
65     await page.mouse.wheel(0, 1000)
66
67
68     # Navigate to a protected route to trigger session
    expiration.
69     frame = context.pages[-1]
70     elem = frame.locator('xpath=html/body/div[2]/div[2]/ul/li
    [16]/a').nth(0)
71     await page.wait_for_timeout(3000); await elem.click
    (timeout=5000)
72
73
74     await asyncio.sleep(5)
75
76     finally:
77         if context:
78             await context.close()
79         if browser:
80             await browser.close()
81         if pw:
82             await pw.stop()
83
84     asyncio.run(run_test())
85

```

Error

The session timeout issue was reported after testing the logout behavior. The user was not redirected to the login page after the session expired, and an access denied message was displayed instead. Task completed.

Cause

The hosting environment may not be properly managing session states, leading to the system not recognizing that the user session has expired and failing to redirect to the login page accordingly.

Fix

Implement session management checks on the server-side to ensure that once a session has expired, any attempt to access secured resources redirects the user to the login page instead of showing an access denied message.

